

Employee API Model, Configuration, and Startup Setup

This section defines the employee entity and DTO classes, configures the database connection, and sets up the ASP.NET Core application services needed to run the Employee Management API.

Employee Entity

The **Employee** entity represents the employee table in the database. Each property maps to a column, and **EmployeeID** acts as the unique identifier for each employee record.

```
namespace EmployeeManagementAPI.Models.Entities
{
    public class Employee
    {
        public Guid EmployeeID { get; set; }
        public required string FirstName { get; set; }
        public required string LastName { get; set; }
        public string? Email { get; set; }
    }
}
```

Employee DTO Classes

DTOs control the data sent to and from the API. The **EmployeeDto** includes the employee ID for responses, while **UpdateEmployeeDto** includes only the fields that can be updated by the user.

```
namespace EmployeeManagementAPI.Models
{
    public class EmployeeDto
    {
        public Guid EmployeeID { get; set; }
        public required string FirstName { get; set; }
        public required string LastName { get; set; }
        public string? Email { get; set; }
    }
}

namespace EmployeeManagementAPI.Models
{
    public class UpdateEmployeeDto
    {
```

```

        public required string FirstName { get; set; }
        public required string LastName { get; set; }
        public string? Email { get; set; }
    }
}

```

Application Settings

The application settings file defines logging behavior, allowed hosts, and the SQL Server connection string used by Entity Framework Core.

```

{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection":
"Server=.;Database=EmployeeDB;Trusted_Connection=true;TrustServerCertificate=true;"
  }
}

```

Program Startup Configuration

The **Program.cs** file registers required services, configures Entity Framework Core to use the SQL Server connection string, enables Swagger during development, and maps controller routes.

```

using EmployeeManagementAPI.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
builder.Services.AddDbContext<ApplicationDbContext>(options =>

```

```

        options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"));

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.Run();

```

Employee Controller

The **EmployeeController** handles API requests for employee records. It supports creating, reading, updating, and deleting employees by using the configured **ApplicationDbContext**.

```

using EmployeeManagementAPI.Data;
using EmployeeManagementAPI.Models;
using EmployeeManagementAPI.Models.Entities;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;

namespace EmployeeManagementAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        private readonly ApplicationDbContext _dbcontext;

        public EmployeeController(ApplicationDbContext dbcontext)
        {

```

```
        this._dbcontext = dbcontext;
    }

    [HttpGet]
    public IActionResult GetAllEmployee()
    {
        var employees = _dbcontext.Employees.ToList();
        return Ok(employees);
    }

    [HttpPost]
    public IActionResult AddEmployee(EmployeeDto employeedto)
    {
        var employeeEntity = new Employee
        {
            FirstName = employeedto.FirstName,
            LastName = employeedto.LastName,
            Email = employeedto.Email
        };

        _dbcontext.Employees.Add(employeeEntity);
        _dbcontext.SaveChanges();
        return Ok(employeeEntity);
    }

    [HttpGet]
    [Route("{id:Guid}")]
    public IActionResult GetEmployeeByID(Guid id)
    {
        var employee = _dbcontext.Employees.Find(id);

        if (employee is null)
        {
            return NotFound();
        }

        return Ok(employee);
    }
}
```

```

[HttpPut]
[Route("{id:Guid}")]
public IActionResult UpdateEmployeeByID(Guid id,
UpdateEmployeeDto employeeDto)
{
    var employee = _dbcontext.Employees.Find(id);

    if (employee is null)
    {
        return NotFound();
    }

    employee.FirstName = employeeDto.FirstName;
    employee.LastName = employeeDto.LastName;
    employee.Email = employeeDto.Email;

    _dbcontext.SaveChanges();

    return Ok(employee);
}

[HttpDelete]
[Route("{id:Guid}")]
public IActionResult DeleteEmployeeByID(Guid id)
{
    var employee = _dbcontext.Employees.Find(id);

    if (employee is null)
    {
        return NotFound();
    }

    _dbcontext.Employees.Remove(employee);
    _dbcontext.SaveChanges();

    return Ok(employee);
}
}

```

Class Management Model, Data, and Controller Setup

This section defines the student entity and DTO classes, configures the Entity Framework Core database context, and implements the API controller used to manage student records.

Student Entity

The **Student** entity represents a student record in the database. Each property maps to a database column, and **StudentId** uniquely identifies each student.

```
namespace ClassManagement.Models.Entities
{
    public class Student
    {
        public Guid StudentId { get; set; }
        public required string FirstName { get; set; }
        public required string LastName { get; set; }
        public string? Email { get; set; }
    }
}
```

Student DTO Classes

DTOs define the data accepted by the API. **StudentDto** is used when creating a student, while **UpdateStudentDto** is used when modifying an existing student.

```
namespace ClassManagement.Models
{
    public class StudentDto
    {
        public required string FirstName { get; set; }
        public required string LastName { get; set; }
        public string? Email { get; set; }
    }
}

namespace ClassManagement.Models
{
    public class UpdateStudentDto
    {
        public required string FirstName { get; set; }
        public required string LastName { get; set; }
    }
}
```

```

        public string? Email { get; set; }
    }
}

```

Database Context

The **ApplicationDbContext** class connects the application to the database and exposes the **Students** table through a **DbSet**.

```

using ClassManagement.Models.Entities;
using Microsoft.EntityFrameworkCore;

namespace ClassManagement.Data
{
    public class ApplicationDbContext : DbContext
    {
        public
        ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) :
        base(options)
        {
        }

        public DbSet<Student> Students { get; set; }
    }
}

```

Student Controller

The **StudentController** provides API endpoints for managing student records. It supports listing all students, retrieving a student by ID, adding a new student, updating an existing student, and deleting a student.

```

using ClassManagement.Data;
using ClassManagement.Models;
using ClassManagement.Models.Entities;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;

namespace ClassManagement.Controllers
{
    [Route("api/[controller]")]
    [ApiController]

```

```
public class StudentController : ControllerBase
{
    private readonly ApplicationDbContext _dbContext;

    public StudentController(ApplicationDbContext dbContext)
    {
        this._dbContext = dbContext;
    }

    [HttpGet]
    public IActionResult GetAllStudents()
    {
        var students = _dbContext.Students.ToList();
        return Ok(students);
    }

    [HttpGet]
    [Route("{Id:Guid}")]
    public IActionResult GetStudentById(Guid Id)
    {
        var student = _dbContext.Students.Find(Id);

        if (student is null)
        {
            return NotFound();
        }

        return Ok(student);
    }

    [HttpPost]
    public IActionResult AddStudent(StudentDto studentDto)
    {
        var student = new Student
        {
            FirstName = studentDto.FirstName,
            LastName = studentDto.LastName,
            Email = studentDto.Email
        };
    }
}
```



```

        _dbContext.Students.Add(student);
        _dbContext.SaveChanges();

        return Ok(student);
    }

    [HttpPut]
    [Route("{Id:Guid}")]
    public IActionResult UpdateStudentById(Guid Id,
UpdateStudentDto updateStudentDto)
    {
        var student = _dbContext.Students.Find(Id);

        if (student is null)
        {
            return NotFound();
        }

        student.FirstName = updateStudentDto.FirstName;
        student.LastName = updateStudentDto.LastName;
        student.Email = updateStudentDto.Email;

        _dbContext.SaveChanges();

        return Ok(student);
    }

    [HttpDelete]
    [Route("{Id:Guid}")]
    public IActionResult DeleteStudentById(Guid Id)
    {
        var student = _dbContext.Students.Find(Id);

        if (student is null)
        {
            return NotFound();
        }
    }

```

```
        _dbContext.Students.Remove(student);  
        _dbContext.SaveChanges();  
  
        return Ok(student);  
    }  
}
```